Manual Técnico

Pizza

Introducción

El presente manual técnico describe la arquitectura, diseño e implementación del videojuego **Pizza Express Tycoon**, desarrollado en el lenguaje de programación **Java** utilizando la biblioteca gráfica **Swing** para la construcción de interfaces gráficas de usuario.

Arquitectura del Sistema

El proyecto se desarrolló utilizando una arquitectura en objetos y POO, lo que permite separar responsabilidades dentro del sistema y facilitar el mantenimiento del código.

La arquitectura del sistema se divide en las siguientes capas:

Capa de Interfaz (interfaz)

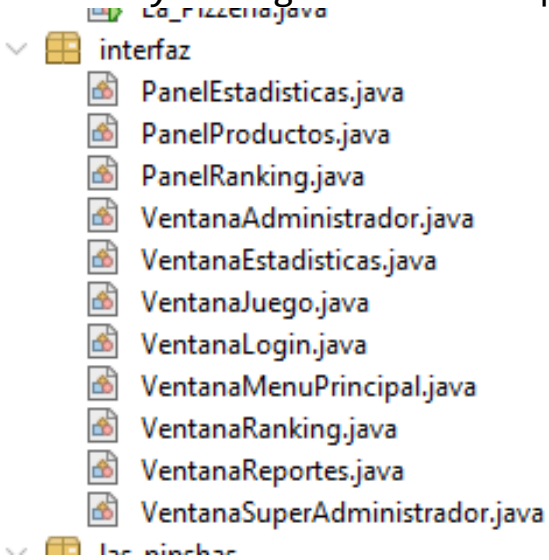
Encargada de gestionar toda la interacción con el usuario.
Incluye las ventanas gráficas desarrolladas con Swing.

Ejemplos de clases:

- VentanaLogin
- VentanaMenuPrincipal
- VentanaJuego
- VentanaAdministrador
- VentanaSuperAdministrador

- VentanaRanking

Estas clases gestionan eventos de botones, presentación de datos y navegación entre pantallas.



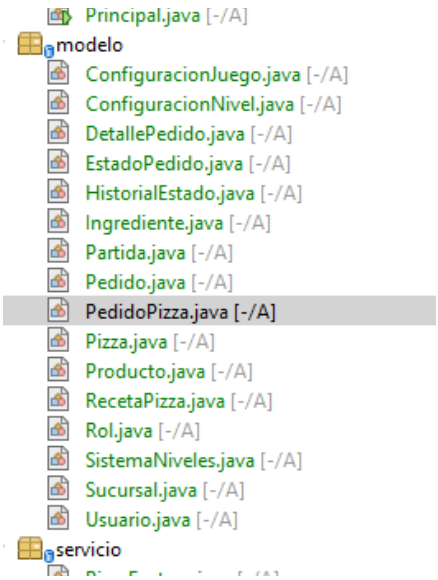
Capa de Modelo (modelo)

Contiene las entidades principales del sistema que representan los datos del videojuego.

Ejemplos:

- Usuario
- Rol
- Sucursal
- Partida
- Pedido
- Ingrediente
- Producto
- PedidoPizza

Estas clases encapsulan atributos y métodos se aplica herencia y son las que representan la información utilizada por el sistema.



Capa de Acceso a Datos (acceso_datos)

Esta capa permite la comunicación con la base de datos mediante objetos DAO (Data Access Object).

Ejemplos:

- UsuarioDAO
- SucursalDAO
- PartidaDAO
- PedidoDAO

Su responsabilidad es ejecutar consultas SQL para:

- Guardar información
- Consultar registros
- Actualizar datos
- Eliminar registros

Capa de Servicios (servicio)

Implemente la lógica de negocio del sistema.

Ejemplo:

ServicioAutenticacion

Esta clase se encarga de validar usuarios y gestionar el proceso de login.

```
9   private final UsuarioDAO usuarioDAO;
0
1   public ServicioAutenticacion() {
2       usuarioDAO = new UsuarioDAO();
3   }
4
5   public Usuario login(String username, String passwordPl
6           throws SQLException, ClassNotFoundException {
7
8       Usuario usuario = usuarioDAO.buscarPorUsername(user
9
0       if (usuario == null) {
1           return null;
2       }
3
4       if (!usuario.isEstado()) {
5           throw new RuntimeException("Usuario inactivo");
6       }
7
8       if (!passwordPlano.equals(usuario.getPassword())) {
9           return null;
0       }
1
2       return usuario;
3   }
```

Base de Datos

El sistema utiliza una base de datos relacional que permite almacenar información sobre usuarios, partidas, pedidos y sucursales.

Las tablas principales incluyen:

Tabla Usuario

Contiene la información de los jugadores y administradores del sistema.

Campos principales:

- id_usuario
- username
- password
- nombre_completo
- estado
- id_rol
- id_sucursal

Ejemplo de inserción inicial:

```
7
public Usuario buscarPorUsername(String username)
9           throws SQLException, ClassNotFoundException {
10
11      String sql = """
12      SELECT u.*,r.id_rol,r.nombre AS rol_nombre,r.descripcion,
13              s.id_sucursal,s.nombre AS sucursal_nombre,s.direccion,s.estado
14      FROM usuario u
15      JOIN rol r ON u.id_rol=r.id_rol
16      LEFT JOIN sucursal s ON u.id_sucursal=s.id_sucursal
17      WHERE u.username=?
18      """;
```

Tabla Rol

Define los distintos tipos de usuario dentro del sistema.

Roles existentes:

1 → ADMIN
2 → ADMIN_TIENDA
3 → JUGADOR

Cada rol tiene acceso a diferentes funcionalidades dentro del sistema.

Tabla Partida

Registra cada sesión de juego realizada por un jugador.

Campos principales:

- id_partida
- id_usuario
- fecha_inicio
- fecha_fin
- nivel_alcanzado
- puntaje_final
- pedidos_correctos
- pedidos_cancelados
- pedidos_tarde

Sistema de Autenticación

El proceso de autenticación se implementa mediante la clase:

`ServicioAutenticacion`

Este servicio valida que el usuario exista en la base de datos y que la contraseña sea correcta.

El proceso funciona de la siguiente manera:

1. El usuario ingresa su nombre y contraseña en `VentanaLogin`.
2. Se envían los datos al servicio de autenticación.
3. El sistema consulta la base de datos mediante `UsuarioDAO`.
4. Si los datos son correctos, el sistema obtiene el rol del usuario.
5. Dependiendo del rol, se abre la ventana correspondiente.

Ejemplo de control de roles:

```
switch (rol) {

    case "JUGADOR":

        new VentanaMenuPrincipal(usuario).setVisible(true);
        break;

    case "ADMIN":

        new VentanaAdministrador(usuario).setVisible(true);
        break;

    case "ADMIN_TIENDA":

        new VentanaSuperAdministrador(usuario).setVisible(tru
        break;

    default:

        JOptionPane.showMessageDialog(this, "Rol no reconocid
}
```

Este mecanismo garantiza que cada usuario tenga acceso únicamente a las funcionalidades permitidas según su rol.

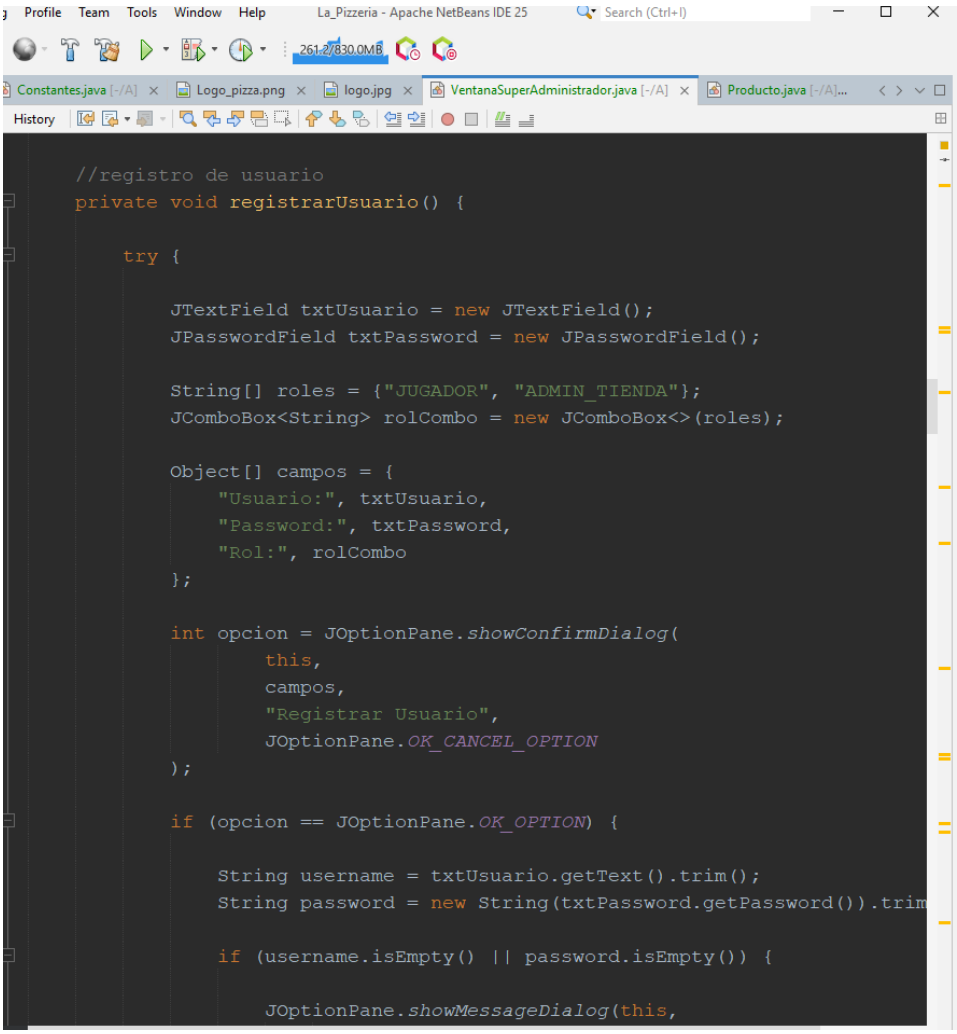
Gestión de Usuarios

El sistema permite registrar nuevos usuarios desde el panel del **Super Administrador.**

El método `registrarUsuario()` permite:

- Crear un usuario
- Asignar un rol
- Validar que el usuario no exista previamente
- Guardar la información en la base de datos

El sistema asegura que cada usuario tenga un rol específico para evitar accesos indebidos a otras funcionalidades del sistema.



Sistema de Niveles

Descripción General

El videojuego incorpora un sistema de progresión por niveles que permite aumentar gradualmente la dificultad durante el desarrollo de una partida.

El jugador inicia siempre en **Nivel 1** y puede avanzar hasta un máximo de **Nivel 3**.

La progresión se realiza automáticamente según el desempeño del jugador durante la partida.

Mecanismo de Progresión

El sistema utiliza dos métricas principales para determinar el avance de nivel:

1. Cantidad de pedidos completados correctamente.
2. Cantidad de puntos acumulados.

Un pedido se considera correctamente completado cuando su estado final es:

- ENTREGADA
- RETIRADA

Reglas de Ascenso de Nivel

Los niveles se incrementan según las siguientes condiciones:

Nivel 1 → Nivel inicial del jugador.

Nivel 2 → Se alcanza cuando el jugador:

- completa **5 pedidos correctamente**, o
- alcanza **500 puntos**.

Nivel 3 → Se alcanza cuando el jugador:

- completa **10 pedidos correctamente**, o
- alcanza **1000 puntos**.

```java
private void sumarPuntos(int puntos) {

    puntaje += puntos;

    lblPuntaje.setText("Puntaje: " + puntaje);

    int pedidosCorrectos = partida.getPedidosCorrectos();

    int nuevoNivel = SistemaNiveles.calcularNivel(puntaje, pedidosCorrectos

    if (nuevoNivel > nivelActual) {

        nivelActual = nuevoNivel;

        JOptionPane.showMessageDialog(this,
                "¡Subiste al NIVEL " + nivelActual + "!");

        lblNivel.setText("Nivel: " + nivelActual);

        actualizarProgresoNivel();
    }

    verificarFinJuego();
}
```

```java
odelo.ConfiguracionNivel;
ava.sql.*;

class NivelDAO {

ic ConfiguracionNivel obtenerPorNivel(int nivel) throws SQLException, ClassN

 String sql = "SELECT * FROM nivel_configuracion WHERE nivel = ?";

 try (Connection conn = ConexionBD.getConexion();
      PreparedStatement stmt = conn.prepareStatement(sql)) {

     stmt.setInt(1, nivel);
     try (ResultSet rs = stmt.executeQuery()) {
         if (rs.next()) {
             return new ConfiguracionNivel(
                     rs.getInt("id_nivel"),
                     rs.getInt("nivel"),
                     rs.getInt("tiempo_base_segundos"),
                     rs.getObject("pedidos_para_subir", Integer.class),
                     rs.getObject("puntos_para_subir", Integer.class));
         }
     }

 } catch (SQLException e) {
     e.printStackTrace();
     //Excepcion para no romper la logica
     throw e;
 }

 return null;
```

Implementación del Sistema

Durante el juego se monitorean constantemente los siguientes datos:

- pedidosCorrectos
- puntajeTotal
- nivelActual

Cuando alguno de los valores alcanza el umbral definido, el sistema actualiza automáticamente el nivel del jugador.

Ejemplo conceptual del algoritmo:

```
Si pedidosCorrectos >= 5 o puntaje >= 500
nivel = 2

Si pedidosCorrectos >= 10 o puntaje >= 1000
nivel = 3
```

Al aumentar el nivel, el sistema incrementa la dificultad del juego mediante:

- reducción del tiempo disponible para completar pedidos
- aumento de la velocidad de generación de pedidos
- mayor presión en la gestión de recursos

```java
package modelo;

public class SistemaNiveles {

    public static final int NIVEL_MAXIMO = 3;

    public static int calcularNivel(int puntaje, int pedidosCorrectos) {

        if (puntaje >= 1000 || pedidosCorrectos >= 10) {
            return 3;
        }

        if (puntaje >= 500 || pedidosCorrectos >= 5) {
            return 2;
        }

        return 1;
    }

}
```

Gestión de Pedidos

Los pedidos dentro del juego siguen un ciclo de estados que representa su proceso de preparación.

Estados del pedido:

RECIBIDA
PREPARANDO
EN_HORNO
LISTA / ENTREGADA

Estados especiales:

CANCELADA
NO_ENTREGADO

Cancelación de pedidos

Un pedido puede ser cancelado por el jugador cuando se encuentra en las etapas iniciales del flujo.

Estados permitidos para cancelar:

- RECIBIDA
- PREPARANDO

Cuando un pedido es cancelado:

- se aplica una penalización de puntos
- el pedido no puede ser modificado nuevamente

Pedidos no entregados

Un pedido se marca automáticamente como **NO_ENTREGADO** cuando el tiempo límite expira antes de completarlo.

Este cambio de estado ocurre automáticamente sin intervención del jugador.

También genera una penalización en el puntaje final.

Sistema de Ranking

El videojuego incorpora un sistema de ranking que permite visualizar el desempeño de los jugadores.

El ranking muestra información como:

- nombre del jugador
- nivel alcanzado
- pedidos completados
- pedidos cancelados
- pedidos no entregados

- puntaje total
- tiempo de juego

Estos datos se obtienen desde la tabla **Partida** y se presentan en una tabla dentro de la ventana `VentanaRanking`.

Panel de Administración

El sistema incluye dos niveles de administración.

Administrador de Tienda

Permite gestionar:

- productos
- ranking de jugadores
- estadísticas de partidas
- exportación de reportes en CSV

```java
package utilidades;

import modelo.Usuario;
import javax.swing.JFileChooser;
import javax.swing.filechooser.FileNameExtensionFilter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;

public class ExportadorCSV {

    public static void exportarRanking(Usuario[] ranking) {
        // 1. Configurar el selector de archivos
        JFileChooser selector = new JFileChooser();
        selector.setDialogTitle("Guardar Ranking");


        FileNameExtensionFilter filtro = new FileNameExtensionFilter("Archivo
        selector.setFileFilter(filtro);


        int resultado = selector.showSaveDialog(null);


        if (resultado == JFileChooser.APPROVE_OPTION) {
            File archivo = selector.getSelectedFile();
            String ruta = archivo.getAbsolutePath();


            if (!ruta.toLowerCase().endsWith(".csv")) {
                archivo = new File(ruta + ".csv");
            }

```

Super Administrador

Tiene acceso completo al sistema.

Puede gestionar:

- sucursales
- usuarios
- asignación de usuarios a sucursales
- ranking global
- estadísticas globales
- configuración de parámetros del juego

Configuración del Juego

El sistema permite modificar ciertos parámetros del juego mediante la clase `ConfiguracionJuego`.

Uno de los parámetros configurables es:

Tiempo base de pedidos.

Esto permite ajustar la dificultad general del videojuego.

Ejemplo:

```
ConfiguracionJuego.TIEMPO_PEDIDO = 30
```

El uso de una base de datos relacional permite almacenar y analizar información sobre partidas y jugadores, mientras que el sistema de roles asegura un control adecuado del acceso a las funcionalidades administrativas.

El sistema de niveles implementado introduce una progresión de dificultad que mejora la experiencia del jugador, incentivando la mejora continua del desempeño durante cada partida.

Este diseño permite futuras mejoras, como la adición de nuevos niveles, mecánicas de juego adicionales o funcionalidades de análisis más avanzadas.

PIZZA-BOT
v5.1
GitHub
PIZZA_BOT
<CODE/>
FLOUR